EXPERIMENT NO: 7
GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK

AIM

To construct and train a Generative Adversarial Network (GAN) using the TensorFlow/Keras framework. The objective is to train the GAN on the MNIST dataset to generate new, synthetic images of handwritten digits that are indistinguishable from the original training data.

PROCEDURE

1. Import the required libraries, including NumPy, Matplotlib, TensorFlow/Keras, the MNIST dataset, and os.
2. Load the MNIST training images and reshape them to 28 × 28 pixels with one channel. Normalize pixel values to the range [-1, 1].
3. Build the Generator, which converts a 100-dimensional random noise vector into a synthetic 28 × 28 grayscale image.
4. Build the Discriminator, which classifies an input image as real or generated.
5. Define binary cross-entropy loss functions for the Generator and Discriminator, and configure both optimizers using Adam with a learning rate of 0.0001.
6. Create a training step that generates fake images, evaluates real and fake images with the Discriminator, calculates losses, and updates both networks.
7. Train the GAN on shuffled MNIST batches for 200 epochs, using a batch size of 256.
8. Every 20 epochs, generate and save a 4 × 4 grid of sample images in the images folder. Generate the final sample grid after training.

CODE

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os

# Suppress TensorFlow warnings for cleaner output
tf.keras.utils.disable_interactive_logging()

# --- Part 1: Dataset Loading and Preprocessing ---
print("--- Part 1: Loading and Preprocessing the MNIST Dataset ---")

(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype('float32')
x_train = (x_train - 127.5) / 127.5 # Normalize to [-1, 1]

print(f"Normalized training data shape: {x_train.shape}")
print("Example of a normalized pixel value:", x_train[0, 0, 0, 0])

# --- Part 2: Building the Generator and Discriminator Models ---
print("\\n--- Part 2: Building the GAN Components ---")
```

```
latent_dim = 100

# Generator
def build_generator():
    model = keras.Sequential(name="generator")
    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(latent_dim,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Reshape((7, 7, 256)))
    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same',
                                     use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same',
                                     use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same',
                                     use_bias=False, activation='tanh'))
    return model

generator = build_generator()
print("\\n--- Generator Model Summary ---")
generator.summary()

# Discriminator
def build_discriminator():
    model = keras.Sequential(name="discriminator")
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same',
                            input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))
    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

discriminator = build_discriminator()
print("\\n--- Discriminator Model Summary ---")
discriminator.summary()

# --- Part 3: Training Setup ---
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)

@tf.function
def train_step(images, latent_dim=latent_dim):
    noise = tf.random.normal([batch_size, latent_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)
    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables)
    generator_optimizer.apply_gradients(
```

```
        zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(
        zip(gradients_of_discriminator, discriminator.trainable_variables))
    return gen_loss, disc_loss

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
    predictions_rescaled = (predictions * 0.5) + 0.5 # Scale back to [0, 1]
    fig = plt.figure(figsize=(4, 4))
    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions_rescaled[i, :, :, 0], cmap='gray')
        plt.axis('off')
    plt.suptitle(f"Epoch {epoch}", fontsize=16)
    if not os.path.exists('images'):
        os.makedirs('images')
    plt.savefig(f'images/image_at_epoch_{epoch:04d}.png')
    plt.show()

# Training parameters
EPOCHS = 200
batch_size = 256
num_examples_to_generate = 16

seed = tf.random.normal([num_examples_to_generate, latent_dim])

train_dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(
    x_train.shape[0]).batch(batch_size)

# Training loop
def train(dataset, epochs):
    print("\\n--- Beginning GAN Training ---")
    for epoch in range(epochs):
        gen_loss_list = []
        disc_loss_list = []
        for image_batch in dataset:
            gen_loss, disc_loss = train_step(image_batch)
            gen_loss_list.append(gen_loss.numpy())
            disc_loss_list.append(disc_loss.numpy())
        avg_gen_loss = np.mean(gen_loss_list)
        avg_disc_loss = np.mean(disc_loss_list)
        print(f"Epoch {epoch + 1}/{epochs} - Generator Loss: {avg_gen_loss:.4f}, "
              f"Discriminator Loss: {avg_disc_loss:.4f}")
        if (epoch + 1) % 20 == 0:
            generate_and_save_images(generator, epoch + 1, seed)
    print("\\n--- Training complete. Generating final images. ---")
    generate_and_save_images(generator, epochs, seed)

# Run training
train(train_dataset, EPOCHS)
```

OUTPUT

The source PDF's OUTPUT section does not contain readable console output or a visible generated-image grid. Therefore, the exact run output cannot be transcribed from the supplied PDF. When the code runs successfully, it prints the dataset shape and a normalized pixel value, displays Generator and Discriminator model summaries, prints the Generator and Discriminator losses for each of the 200 epochs, and displays generated digit images.

```
--- Part 1: Loading and Preprocessing the MNIST Dataset ---
Normalized training data shape: (60000, 28, 28, 1)
Example of a normalized pixel value: [value depends on the first MNIST image pixel]


--- Part 2: Building the GAN Components ---
--- Generator Model Summary ---
[Keras Generator model summary printed here]
--- Discriminator Model Summary ---
```

```
[Keras Discriminator model summary printed here]
--- Beginning GAN Training ---
Epoch 1/200 - Generator Loss: [runtime value], Discriminator Loss: [runtime value]
...
Epoch 200/200 - Generator Loss: [runtime value], Discriminator Loss: [runtime value]
--- Training complete. Generating final images. ---
Generated image grids are displayed and saved as images/image_at_epoch_0020.png, ...
images/image_at_epoch_0200.png.
```

RESULT

The Generative Adversarial Network (GAN) was implemented for training on the MNIST dataset. The Generator creates synthetic images, while the Discriminator learns to distinguish real samples from generated samples. The experiment is intended to demonstrate the GAN's ability to learn patterns in handwritten digits and generate synthetic outputs.